

Manual Tecnico
Daren Uziel Zapeta Molina
202132253

1. Introducción

En el siguiente documento se describirán a detalle la implementación del analizador léxico del lenguaje PromptZal, desarrollado totalmente en java. Toda la implementación se hizo de forma manual, recorriendo la entrada carácter por carácter.

El programa recibe la ruta del archivo .pz y la de la carpeta donde se quiere guardar el documento HTML, reconoce los tokens y errores léxicos, y los muestra en tablas.

2. Arquitectura del lexer

El analizador está organizado en cuatro paquetes de java, cada uno con una responsabilidad, separados de la siguiente forma:

Package/Clase	Descripción
Main.Main	La clase Main maneja todas las interacciones por consola: pide rutas, imprime los resultados y genera los HTML.
PromptZalLenguaje.TipoToken	Esta clase Enum define las categorías del lenguaje PromptZal.
Registros.RegistroToken	Esta clase define la tupla de información de cada token reconocido.
Registros.RegistrosError	Esta clase define la tupla de información de los errores léxico que encuentre.
AnalizadorLexico.Lexer	El analizador lexico en si. Recorre todo el archivo caracter por caracter y generando la lista de tokens y errores.

2.1 Flujo general

1. Main pide al usuario la ruta de un archivo .pz, validando antes que existían estas rutas o que sean válidas.
2. Main pide la ruta de una carpeta donde guardar el reporte HTML, validando que sea una carpeta real.

3. procesarArchivo() pasa a un string el contenido del documento y crea un lexer con ese texto.
4. Se invoca lexer.analizar(), que recorre todo el texto esperando en tokens y errores en las listas.
5. Se obtienen ambas listas y se imprimen en consola, además genera el reporte HTML.
6. Se muestra un menú para continuar o terminar el programa.

3. Clase TipoToken

TipoToken es una clase enum que define las distintas categorías léxicas del lenguaje PromptZal que se pueden reconocer como token:

```
public enum TipoToken{  
    DIRECTIVA, PALABRA_RESERVADA, COMANDO_IA, CONECTOR, IDENTIFICADOR,  
    CADENA, ENTERO, DECIMAL, OP_ASIGNACION, OP_CONCATENACION,  
    LLAVE_ABRE, LLAVE_CIERRA, PARENTESIS_ABRE, PARENTESIS_CIERRA,  
    COMA, FUNCION  
}
```

4. RegistroToken y RegistroError

Las clases de registros no contienen lógica, sólo almacenan la información analizada por el lexer.

```
public class RegistroToken {  
  
    private String lexema;  
    private TipoToken tipo;  
    private int fila;  
    private int columna;  
  
    public RegistroToken(String lexema, TipoToken tipo, int fila, int columna) {  
        this.lexema = lexema;  
        this.tipo = tipo;  
        this.fila = fila;  
        this.columna = columna;  
    }  
}  
  
public class RegistroError {  
  
    private String lexema;  
    private String descripcion;  
    private int fila;  
    private int columna;  
  
    public RegistroError(String lexema, String descripcion, int fila, int columna) {  
        this.lexema = lexema;  
        this.descripcion = descripcion;  
        this.fila = fila;  
        this.columna = columna;  
    }  
}
```

5. Analizador léxico

La clase que contiene toda la lógica se encarga de analizar el texto que está en un arreglo de caracteres (texto del .pz), además de ir llenando las List de tokens y de errores, sin detener el análisis en ningún momento.

5.1 Atributos

Atributos	Descripción
char[] texto	aquí se guarda el texto del archivo .pz en un arreglo de caracteres
int posicion	indica la posición actual en el arreglo
int fila, int columna	indican la fila y la columna actual en el texto
List tokens	lista donde se almacenan los tokens reconocidos
List errores	lista donde se almacenan los errores encontrados

5.2 Métodos del arreglo

Método	Que hace
finDeArchivo()	devuelve true si la posición ya superó al tamaño del arreglo, indicando que ya no hay más caracteres que analizar.
actual()	devuelve el carácter de la posición actual en el arreglo, sin pasar al siguiente carácter.
siguiente()	devuelve el carácter en la posición siguiente sin avanzar, sirve para tokens que necesiten saber más, además de la posición actual.
avanzar()	Avanza al siguiente token, además incrementa la posición y actualiza fila/columna.

5.3 Método clasificadores

Método	Que hace
esDigito(char c)	devuelve un true si el caracter que recibe es un dígito
esLetra(char c)	devuelve un true si el caracter que recibe es una letra
esInicioIdentificador(char c)	devuelve true si el carácter cumple como inicio de un identificador
esParteIdentificador(char c)	devuelve true si el carácter cumple para ser la continuación de un identificador

5.4 método base del analizador

Este método es el encargado de analizar el carácter en ese momento, siguiendo un orden para identificar de qué tipo de token se trata y si existe algún error, el orden es el siguiente:

1. Espacio en blanco y saltos con Tab
2. Saltos de línea
3. // comentario de una línea
4. /* comentario de varias líneas
5. @ directivas
6. " cadenas de texto
7. dígitos
8. identificadores
9. conector "->"
10. operadores
11. delimitadores
12. si no es ninguno de los anteriores es tomado como error

```
//define el tipo de token que es
private void leerToken() {
    char c = actual();

    if (c == ' ' || c == '\t') {
        avanzar();
        return;
    }

    if (c == '\n') {
        avanzar();
        return;
    }

    if (c == '/' && siguiente() == '/') {
        leerComentarioDeLinea();
        return;
    }

    if (c == '/' && siguiente() == '*') {
        leerComentarioDeBloque();
        return;
    }
}
```

5.5 métodos especiales para un tipo de token

Método	descripción
leerComentarioDeLinea()	este método se invoca si se detecta dos “/”, lo que indica que es un comentario de línea, genera ningún token solo ignora el resto de la línea
leeComentarioDeBloque()	Este método se invoca si se detecta “/” <i>indicando que es un comentario de varias líneas. no genera ningún token solo ignora hasta encontrar un “*/”</i>
leerDirectiva()	Este método se invoca cuando se encuentra un “@”. Lo toma como una directiva y valida que todos los tokens siguientes sean letras, si al final no forma alguna de las directivas conocidas se toma como error.
leerCadena()	este método se invoca cuando se encuentra un “ en el texto, indicando que los caracteres siguientes forman una cadena de texto, si no encuentra un cierre con “ lo toma como error
leerNumero()	Este método se invoca cuando detecta que el carácter que está analizando es un número. Este método solo maneja número y punto si se trata de un número decimal, si detecta alguna irregularidad lo toma como error.
leerIdentificadorOPalabraClave()	Este método se invoca cuando el carácter que está analizando es una letra o un guión bajo. Este método analiza primero si se trata de una palabra reservada, comando IA, conector, función, si no es ninguna de las anteriores lo toma como identificador.

5.6 Método Analizar()

Este método es el que hace todo, usando un while que tiene como condición el método finDeArchivo(), debe recorrer caracter por caracter y pasarlo por la clase que analiza el carácter actual, si el archivo se acaba el ciclo while se cierra.

```
public void analizar() {  
    while (!finDeArchivo()) {  
        leerToken();  
    }  
}
```

Este es el método que se invoca desde el main para que analice el texto.